

UNIVERSIDAD DEL VALLE DE GUATEMALA

ICC30672020262 – Redes

Sección 20

Jorge Andrés Yass Coy



Laboratorio #1

JAVIER EDUARDO ESPAÑA PACHECO #23361

GUATEMALA, 13 de julio de 2026

Introducción

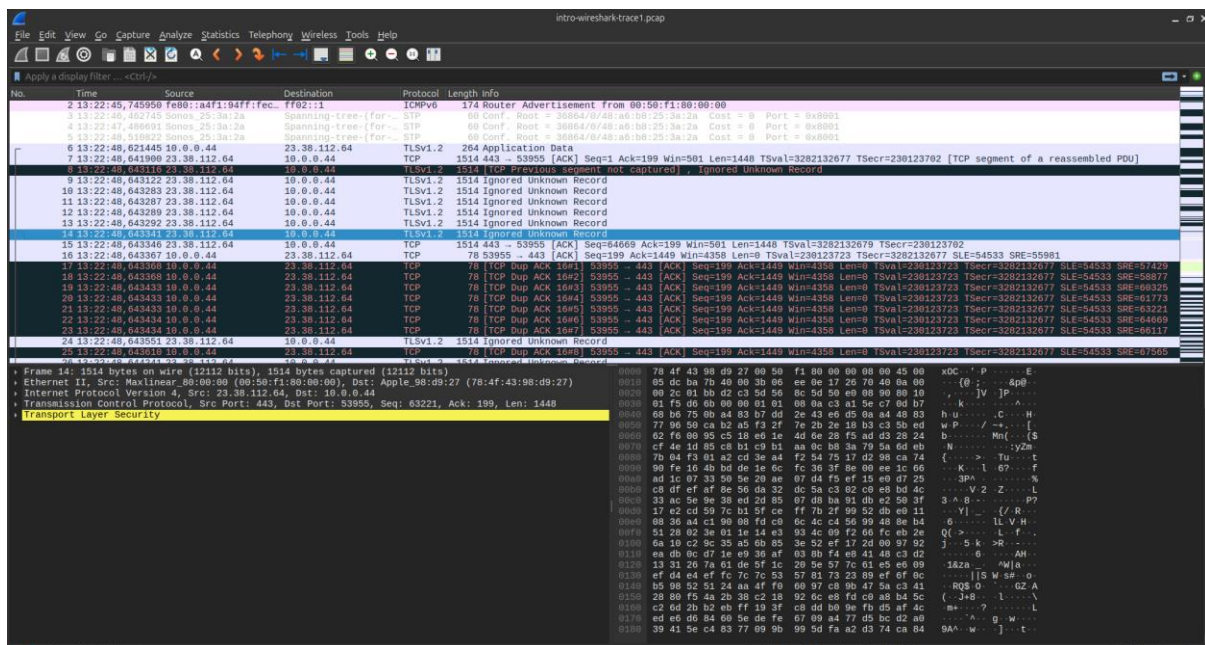
El laboratorio explora los fundamentos de las comunicaciones digitales y el análisis de tráfico de red. En la primera parte, se compararon los esquemas Morse y Baudot mediante transmisiones en tiempo real y grabadas, y se simuló un sistema de conmutación de mensajes para entender sus ventajas y limitaciones.

En la segunda parte, se utilizó Wireshark para capturar y analizar paquetes de red. Se personalizó el entorno, se configuró una captura con ring buffer y se analizó tráfico HTTP, permitiendo observar el funcionamiento de los protocolos y la importancia del análisis de paquetes para diagnóstico de redes.

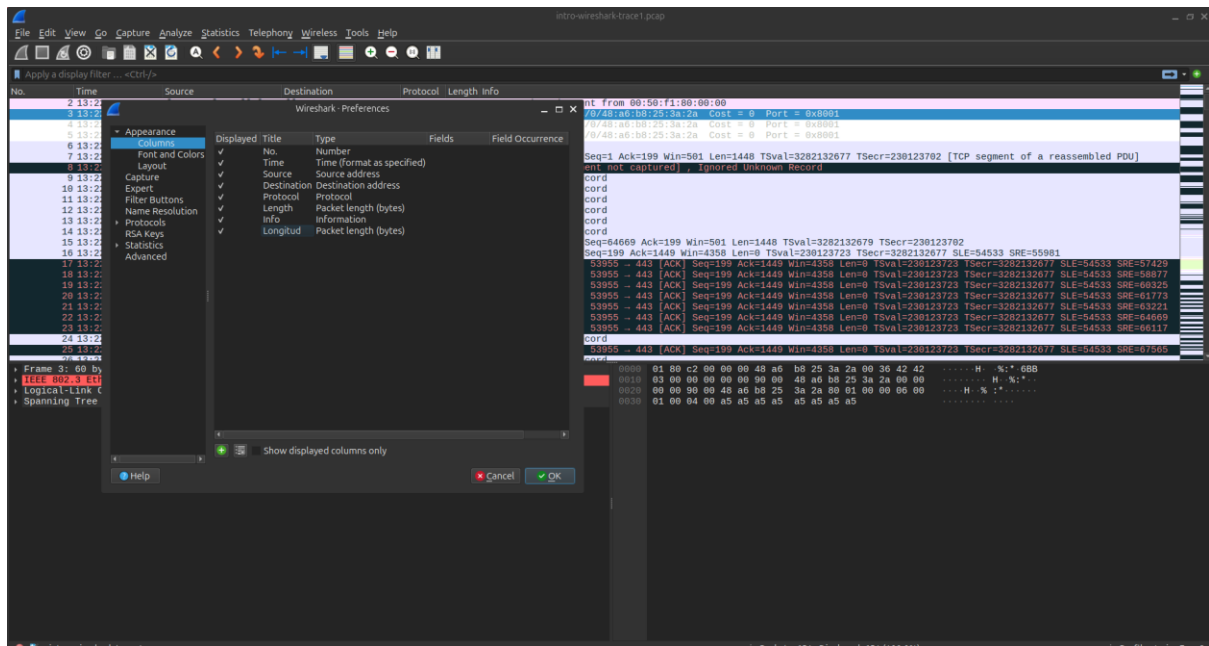
3.4 Personalización del entorno

Se siguió el procedimiento indicado para configurar el entorno de Wireshark:

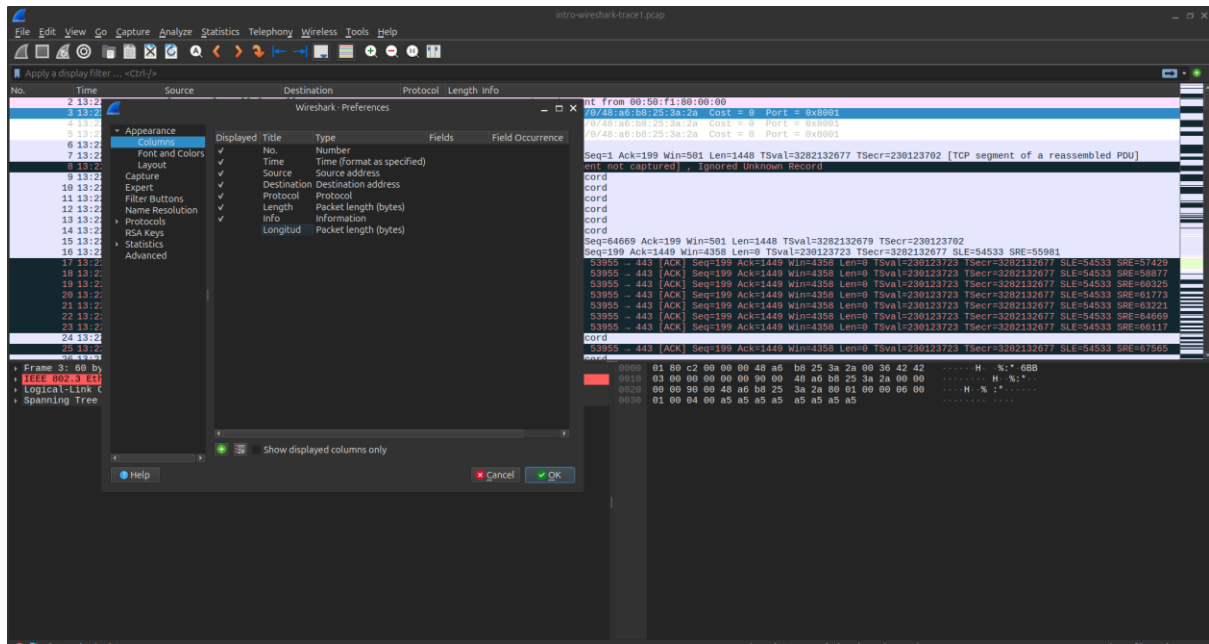
- Perfil personalizado con el nombre “**Javier España**”.



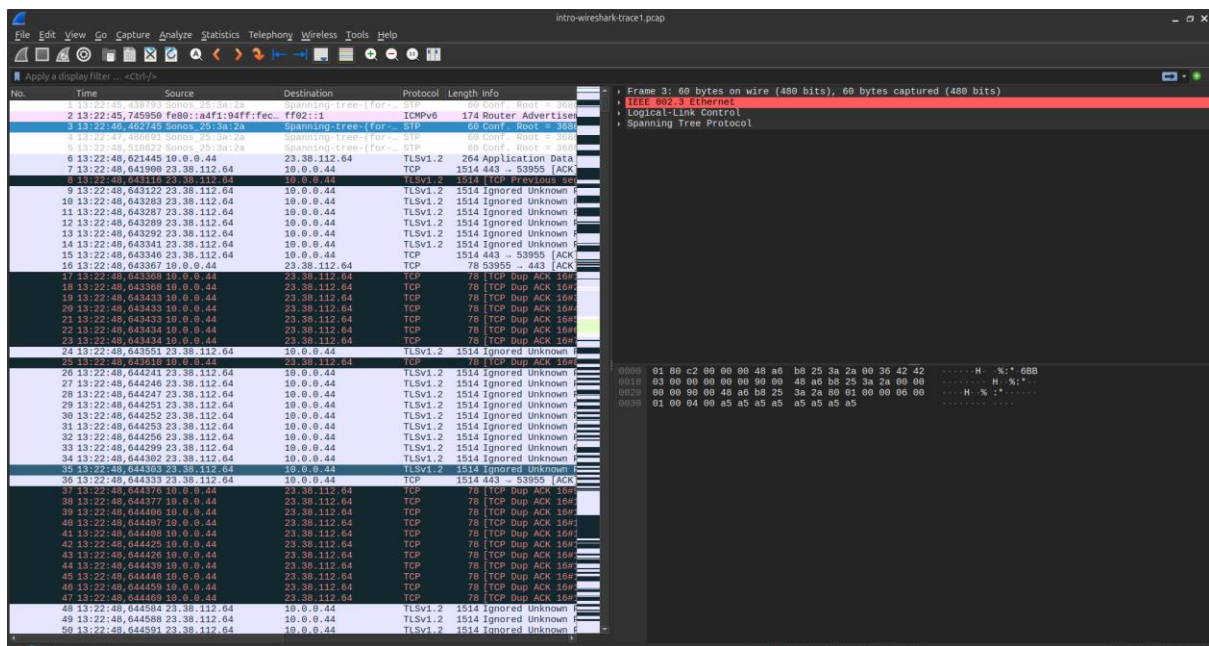
- Cambió el formato de tiempo a *Time of Day*.



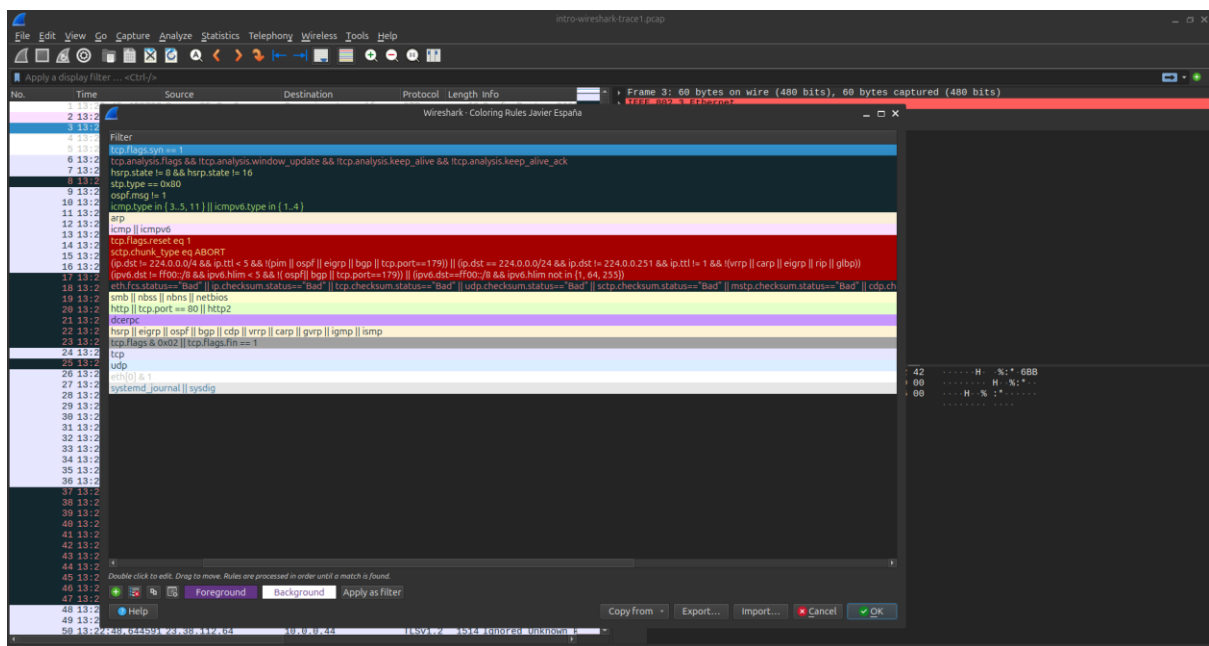
- Columna para la longitud del protocolo y oculté la columna “Longitud” predeterminada.



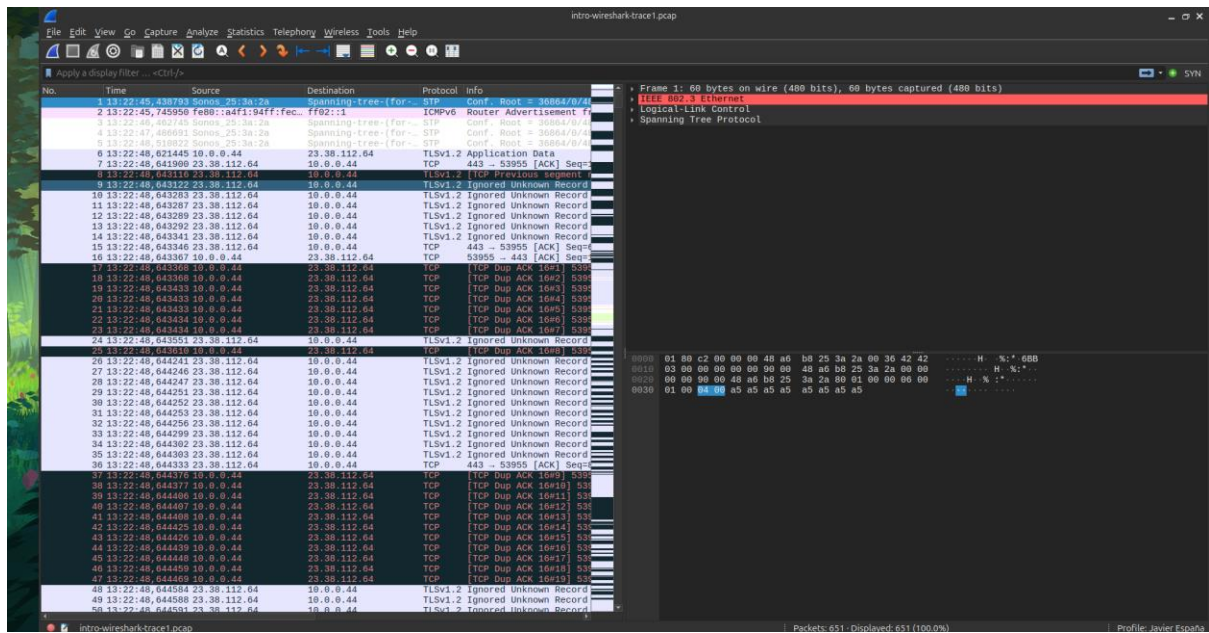
- Modifiqué el layout



- Se creó una regla de color para resaltar paquetes TCP con la bandera SYN igual a 1. La expresión utilizada fue `tcp.flags.syn == 1`.



- Se añadió un botón de filtro en la barra superior con la misma expresión, etiquetado como “SYN” (Figura 5).



3.5 Configuración de captura con ring buffer

Comando ifconfig


```

ifconfig
br-55aa27621bfa: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.16.1.1 netmask 255.255.255.0 broadcast 172.16.1.255
    ether 52:b0:5c:63:74:13 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.16.0.1 netmask 255.255.255.0 broadcast 172.16.0.255
    ether ce:f4:2a:11:5e:ef txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 874 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Bucle local)
    RX packets 2147 bytes 203647 (203.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2147 bytes 203647 (203.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth6f82c78: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    ether f2:75:ee:ab:ae:a1 txqueuelen 0 (Ethernet)
    RX packets 38274 bytes 4021210 (4.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 35719 bytes 4697983 (4.6 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth85f1758: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    ether 7e:15:f6:8a:08:39 txqueuelen 0 (Ethernet)
    RX packets 14681 bytes 2248461 (2.2 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 15207 bytes 1090297 (1.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ethbec28fa: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    ether 4e:ca:9f:e0:01:d8 txqueuelen 0 (Ethernet)
    RX packets 1944 bytes 164303 (164.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8909 bytes 22172817 (22.1 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ethc5597a7: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    ether fa:d4:85:cb:51:d5 txqueuelen 0 (Ethernet)
    RX packets 39651 bytes 3546401 (3.5 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 44233 bytes 5877341 (5.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.100.13.57 netmask 255.255.224.0 broadcast 10.100.31.255
    inet6 fe80::351:4a0:7194:b687 prefixlen 64 scopeid 0x20<link>
    ether e0:d4:64:b3:68:7f txqueuelen 1000 (Ethernet)
    RX packets 2439087 bytes 1067900601 (1.0 GB)
    RX errors 0 dropped 1173 overruns 0 frame 0
    TX packets 139438 bytes 54503167 (54.5 MB)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

```

Observaciones:

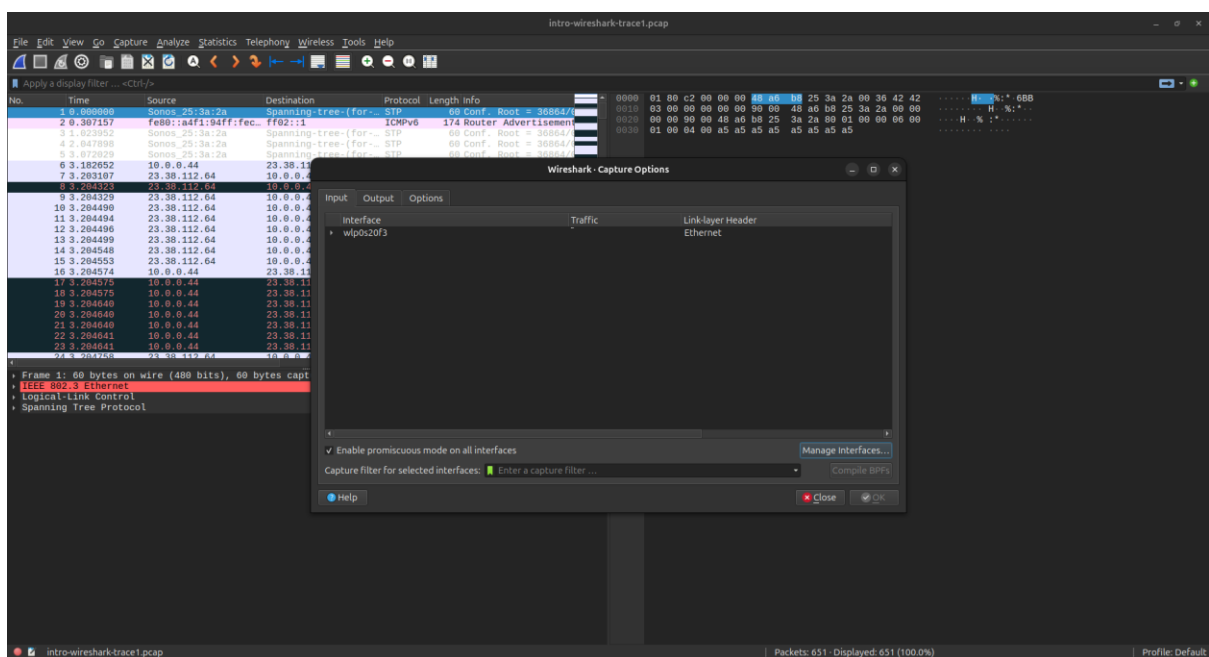
- La interfaz wlp0s20f3 es la tarjeta Wi-Fi activa, con dirección IP 10.100.13.57 y máscara de red 255.255.224.0. Presenta tráfico entrante y saliente (RX/TX), lo que confirma que es la interfaz real de conexión a Internet.
- La interfaz lo (loopback) tiene dirección 127.0.0.1 y se usa para comunicaciones internas del sistema.
- Las interfaces docker0, br-55aa27621bfa y las veth* son bridges y dispositivos virtuales creados por Docker para la comunicación con contenedores; no tienen tráfico relevante en este contexto.

- También aparecen interfaces como bluetooth0 y nftlog, que corresponden a servicios del sistema y no se utilizan para la captura.

Conclusión: La interfaz adecuada para la captura de paquetes es wlp0s20f3.

Desactivación de interfaces virtuales en Wireshark

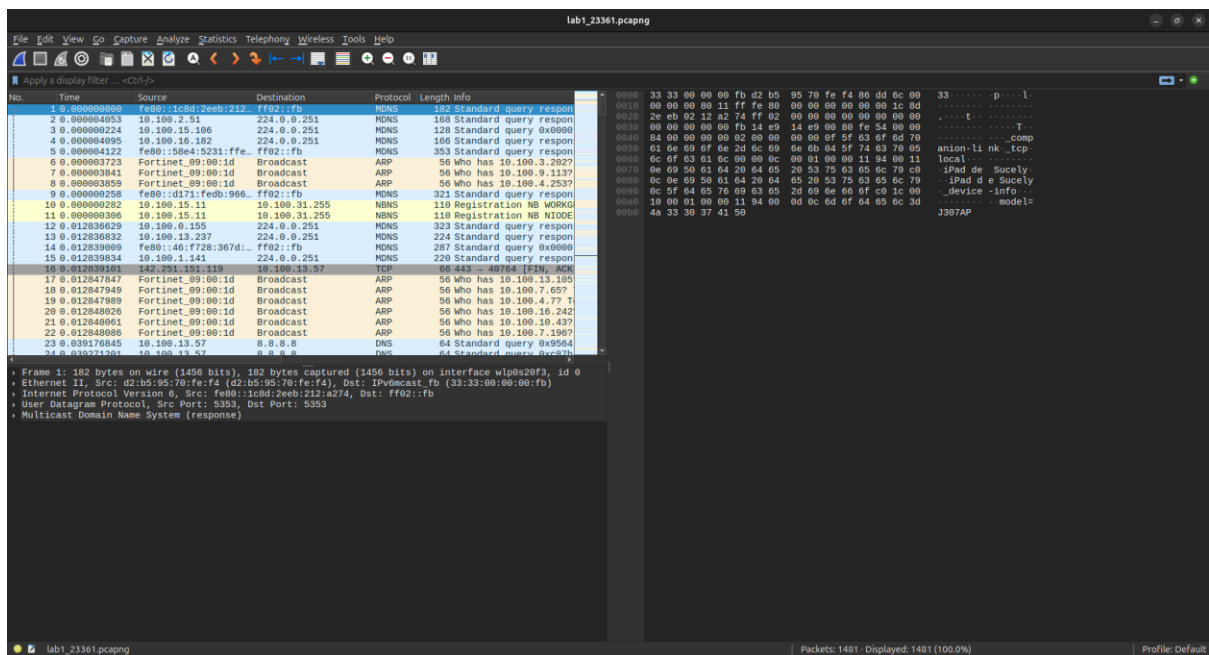
Desde la ventana Capture → Options → Manage Interfaces se desmarcaron todas las interfaces excepto wlp0s20f3, para evitar ruido visual y centrarse únicamente en el tráfico de la red Wi-Fi. La lista simplificada se muestra en la siguiente captura de pantalla.



Configuración del ring buffer

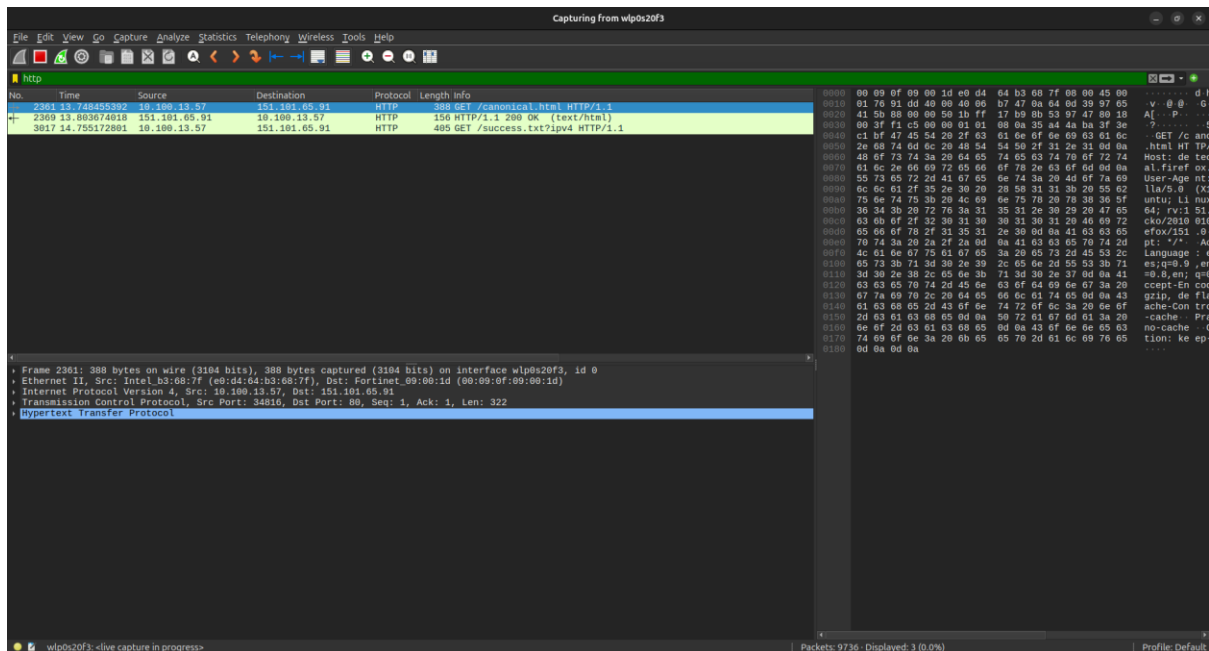
Se configuró la captura con ring buffer a través de la interfaz gráfica de Wireshark:

- **Interfaz:** wlp0s20f3
- **Nombre base del archivo:** lab1_23361.pcapng
- **Tamaño por archivo:** 5 MB
- **Número máximo de archivos:** 10



3.6 Análisis de paquetes HTTP

Se realizó una captura de tráfico mientras se accedía a la dirección <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>. Luego se aplicó el filtro http para aislar las peticiones y respuestas.



a. ¿Qué versión de HTTP está ejecutando su navegador?

HTTP/1.1 porque en la solicitud GET enviada por el cliente (paquete 2361) se lee GET /canonical.html HTTP/1.1.

b. ¿Qué versión de HTTP está ejecutando el servidor?

HTTP/1.1. porque la respuesta del servidor (paquete 2369) muestra el campo HTTP/1.1 200 OK.

c. ¿Qué lenguajes indica el navegador que acepta al servidor?

El navegador indica que acepta los siguientes idiomas: es-ES, es; q=0.9, en-US; q=0.8, en; q=0.7. Al expandir la sección HTTP de la solicitud GET, se puede observar el encabezado Accept-Language con el contenido mencionado. La interpretación de los valores q indica la preferencia del navegador: el idioma es-ES tiene la prioridad más alta (0.9), seguido de es (0.9), en-US (0.8) y en (0.7).

d. ¿Cuántos bytes de contenido fueron devueltos por el servidor?

El tamaño del paquete capturado es de 156 bytes, pero ese valor incluye las cabeceras de enlace, IP, TCP y HTTP. El contenido real se especifica en el campo Content-Length de la respuesta HTTP.

e. En el caso de un problema de rendimiento, ¿en qué elementos de la red convendría “escuchar” los paquetes? ¿Es conveniente instalar Wireshark en el servidor? Justifique.

Puntos de captura recomendados:

- En el cliente (origen): para descartar problemas locales (Wi-Fi, pila TCP/IP).
- En el router o gateway local: para verificar si hay degradación en el primer salto.
- En el servidor o balanceador de carga: para medir el tiempo de procesamiento interno y latencia de respuesta.

Instalación de Wireshark en el servidor:
No es conveniente instalar la versión gráfica de Wireshark en un servidor de producción.

Justificación:

- La GUI consume recursos de CPU y memoria que pueden afectar el rendimiento del servicio principal.
- Aumenta la superficie de ataque al instalar componentes gráficos y bibliotecas adicionales.

- La práctica recomendada es utilizar herramientas de línea de comandos como tcpdump o tshark para capturar el tráfico de forma ligera, guardar el archivo .pcap y transferirlo a una máquina local para su análisis visual con Wireshark.

Discusión

La primera parte del laboratorio permitió experimentar de primera mano las dificultades de la comunicación digital. El código Morse resultó más fácil de recibir porque los sonidos son distinguibles (puntos cortos vs. rayas largas), mientras que el código Baudot (5 bits) fue más propenso a errores porque todos los bits suenan igual y solo se diferencian por su presencia o ausencia. Un error en un solo bit cambia completamente el carácter decodificado.

La transmisión mediante notas de voz introdujo el problema del "paquete" estático: no hay retroalimentación inmediata, por lo que los errores no se pueden corregir en tiempo real. Esto refleja lo que ocurre en redes modernas con paquetes corruptos o perdidos.

El ejercicio del conmutador mostró que centralizar la comunicación tiene ventajas (menos conexiones directas, posibilidad de priorizar tráfico) pero también desventajas críticas: el conmutador es un punto único de falla y puede saturarse si hay mucho tráfico. Agregar más conmutadores distribuye la carga y crea redundancia, pero requiere mecanismos de enrutamiento más complejos.

En la segunda parte, la personalización de Wireshark resultó muy útil para agilizar el análisis. La regla de color para paquetes SYN y el botón de filtro permiten identificar rápidamente intentos de conexión. El ring buffer demostró ser práctico para capturas largas sin llenar el disco.

El análisis HTTP evidenció que la información relevante (como Accept-Language o Content-Length) no es visible en la lista de paquetes, sino en los detalles. También se confirmó que el overhead de cabeceras consume ancho de banda: aunque el paquete mida 156 bytes, el contenido real puede ser mucho menor.

Finalmente, se concluyó que instalar Wireshark en servidores de producción no es recomendable por el consumo de recursos; es mejor usar herramientas CLI como tcpdump y analizar las capturas después en una máquina local.

Conclusiones

- La codificación y decodificación de mensajes requiere sincronización y protocolos claros; el código Morse tiene más redundancia que Baudot, lo que lo hace más robusto.

- La conmutación es fundamental para escalar redes, pero introduce riesgos de congestión y puntos de falla.
- Wireshark es una herramienta educativa y profesional esencial que permite comprender el funcionamiento de los protocolos a nivel de paquetes.
- En entornos productivos, se deben usar herramientas ligeras como tcpdump para capturar y luego analizar offline.

Referencias

- Wireshark Foundation. (s.f.). *Wireshark User's Guide*.
<https://www.wireshark.org/docs/>